Cours
Langage C

PLAN GENERAL

0 - Introduction 3

1 - Notion fondamentales 6

2 - Types de base 15

3 - Opérateurs et expressions 20

4 - Instructions 42

5 - Bibliothèque d'Entrée/Sortie standard (1/2) 48

6 - Types agrégés 54

7 - Fonctions 62

8 - Bibliothèque d'Entrée/Sortie standard (2/2) 68

9 - Gestion mémoire 75

10 - Chaînes de caractères 88

11 - Le préprocesseur 97

0 - Introduction

1 - Généralités

Le C est un langage de programmation conçu et mis en œuvre en 1972 par Dennis Ritchie, pour les Laboratoires Bell dans le New Jersey. Il dérive du langage B et est connu pour son étroite implication avec le système d'exploitation UNIX. Sa large diffusion qui accompagne donc celle d'UNIX est une garantie d'une normalisation certaine de son implantation.

Une première définition rigoureuse a été publiée en 1978. De nombreux compilateurs ont alors vu le jour, en se fondant sur cette définition, quitte à l'assortir parfois de quelques extensions. Ce succès international du langage a conduit à sa normalisation, d'abord par l'ANSI (American National Standard Institute), puis par l'ISO (International Standard Organization), plus récemment par le CEN (Comité Européen de Normalisation) et enfin, en 1994, par l'AFNOR (Association Française de NORmalisation). En fait, toutes ces normes sont identiques et l'usage veut qu'on parle de C ANSI.

Le C est un langage conçu pour de nombreuses utilisations et est caractérisé par une économie d'expressions, par des structures de données, par un ordonnancement moderne des instructions, ainsi que par un ensemble très complet d'opérateurs. Il dispose par ailleurs de la plupart des structures de contrôle qui constituent les bases de la programmation structurée.

Ce langage, comme le PASCAL auquel on l'a souvent comparé, voire opposé, est avant tout un langage qui permet au programmeur de définir les objets ou structures de données propres au problème qu'il doit traiter, sans avoir à les couler dans le moule d'un nombre restreint de types élémentaires. On trouve, dans ce langage, les moyens de contrôler la représentation interne des types de données, ainsi que leur implantation et leur durée de vie en mémoire.

0 - Introduction

Le langage C n'est pas spécialisé dans un champ particulier d'applications. Ainsi, l'absence de restrictions et son côté général le rendent plus adapté et plus efficace dans beaucoup de cas que des langages supposés plus puissants. Il n'est en outre lié à aucune architecture matérielle et permet d'écrire, avec un peu de soin, des programmes portables, c'est à dire des programmes pouvant être utilisés sur une large gamme de matériels, sans être modifiés. Ce qui n'est pas le cas pour de nombreux autres langages.

2 - Aperçu général du langage C

Un programme C est constitué d'une suite de déclarations et d'une suite d'instructions. Les déclarations servent à définir les objets du programme. Quant aux instructions, elles déterminent les actions qui seront effectuées sur ces mêmes objets.

Les objets incluent variables et fonctions. Quand un objet est défini, telle une variable, il lui est associé un identificateur, un type ainsi qu'une durée de vie mémoire. L'identificateur sert à désigner cet objet de manière à ce qu'il puisse être référencé plus tard. Le type quant à lui définit ses caractéristiques opérationnelles et, dans certains cas, indique la notation utilisée pour référencer cet objet.

Il est possible, en C, de définir de nouveaux types de données en utilisant les structures et les unions. Une structure est un rassemblement d'objets d'éventuellement différents types. Une union fournit pour sa part la possibilité de référencer un même objet de différentes manières à différents moments.

0 - Introduction

Le C est relativement différent des autres langages en ce sens qu'il n'a que des fonctions. La notion de procédure est réalisée en déclarant une fonction ne retournant aucun résultat.
Les fonctions en C ont des similarités avec les fonctions mathématiques : une fonction est une entité de calcul qui s'applique à des arguments et génère un résultat. Cependant elles s'en différencient par le fait qu'elles peuvent induire des effets de bord.
Les fonctions peuvent être appelées récursivement mais ne peuvent être internes à d'autres fonctions.

3 - Inconvénients du langage C

Les principaux inconvénients que l'on rencontre dans ce langage sont :

- non lisibilité du langage
- langage à mots clés
- aucun opérateur sur les chaînes de caractères
- mauvaise priorité de certains opérateurs
- pas d'allocation dynamique
- aucune vérification d'indices, de types, d'arguments, etc...
- aucune opération standard d'entrée/sortie
- entrées/sorties formatées comme en ForTran
- pas d'accès fichier séquentiel indexé

1 - Notions fondamentales

1 - Conventions lexicales

Le langage C comporte cinq différentes classes d'éléments :

- les mots réservés
- les identificateurs
- les constantes
- les opérateurs
- et les autres séparateurs de lexème.

Les séparateurs de lexème sont :

- le blanc
- le caractère de tabulation horizontale \t
- le caractère LF (Line Feed ou saut de ligne) \n
- le caractère CR (Carriage Return ou retour chariot) \r
- commentaires

Les commentaires commencent par `/*` et se terminent par `*/`

1 - Notions fondamentales

2 - Les mots réservés

```
asm (*)
auto
break
case
char
const
continue
default
do
double
entry (*)
else
enum
extern
float
for
fortran (*)
goto
if
int
long
return
sizeof
static
struct
switch
typedef
union
unsigned
void
volatile
while
short
```

(*) pas sur tous les compilateurs

1 - Notions fondamentales

3 - Les identificateurs

Ils sont faits de lettres, de chiffres et du caractère '_' (tiret souligné). Ils doivent impérativement commencer par une lettre. Le caractère '_' est considéré comme une lettre.

Les lettres majuscules sont différentes des minuscules.

La reconnaissance s'effectue sur les 8 premiers caractères, mais on peut en mettre davantage.

La restriction sur la longueur peut être encore plus forte en ce qui concerne les identificateurs exportés par un module, c'est à dire ceux qui sont traités par l'éditeur de liens.

4 - Les constantes

Il est très maladroit, dans un programme, de faire apparaître des nombres comme `118` ou `3.14` sans préciser ce qu'ils représentent. Ils ne constituent qu'une très faible information pour ceux qui auraient à lire le programme ou à le modifier. Il est en effet bien plus judicieux de représenter ces nombres par un identificateur. Pour cela, le C fournit une directive `#define` qui permet de définir un nom symbolique et de lui associer une valeur.

```
#define nom_symbolique valeur
```

Exemple :

```
#define PI 3.1415
```

1 - Notions fondamentales

4 - 1 - Les constantes entières

Une constante entière est une suite de chiffres. Les nombres entiers sont supposés être représentés par le système de numération décimal, à moins qu'ils ne soient explicitement exprimés en octal ou en hexadécimal.
Un nombre représenté en octal doit impérativement commencer par le chiffre `0`. Les chiffres `8` et `9` sont acceptés. Ainsi, la constante octale `080` correspond à la valeur décimale `64`.
Un nombre représenté en hexadécimal doit impérativement commencer par `0x` ou `0X`. Les nombres hexadécimaux sont considérés comme non signés, à moins qu'ils ne soient exprimés sur 32 bits avec le MSB (Most Significant Bit) à 1.

Exemples :

```
628
0114
0x1B7F
0235L
```

1 - Notions fondamentales

4 - 2 - Les constantes caractères

Une telle constante est un simple caractère entre apostrophes.
Si un caractère n'est pas imprimable, le C autorise une notation à l'aide d'un caractère escape qui est `\` (antislash).
En ce qui concerne les autres caractères non imprimables, on dispose de la notation `'\n'` ou n est un nombre exprimé en octal (code ASCII).

Exemples :

```
'a'
'\n'                // saut de ligne
'\t'                // tabulation horizontale
'\b'                // espace arrière
'\r'                // retour chariot
'\f'                // saut de page
'\\'                // antislash
'\''                // apostrophe
'\000'              // NUL
'\002'              // ETX
```

4 - 3 - Les constantes flottantes

Une telle constante a une partie entière, un point décimal, une partie décimale et un éventuel exposant. L'exposant consiste en la lettre **e** ou **E** suivie d'une suite de chiffres. L'exposant ou le point décimal peuvent être omis, mais pas les deux à la fois.

Exemples :

```
0.0
3.14159
.235
1.5E-3
```

4 - 4 - Les constantes chaînes

Une constante chaîne est une suite de caractères entourés de guillemets (`"`). Toutes les notations de caractères non imprimables sont utilisables dans les chaînes.

Exemples :

```
"Ceci est une chaîne"
"Si on m'imprime,\nje serai sur\n3 lignes"
```

Le caractère de continuation de ligne est `\`.
Une chaîne de caractères dans le langage C a le type de tableau de caractères et est une donnée statique.
La valeur initiale d'une chaîne est celle des caractères se trouvant entre guillemets. Cependant, le compilateur place toujours un octet nul en fin de chaîne de manière à en indiquer la longueur.
Toute constante chaîne de caractères est rangée à un endroit différent, même si deux chaînes sont identiques. En d'autres termes, le compilateur ne partage pas la mémoire pour le rangement de chaînes de caractères.

1 - Notions fondamentales

4 - 5 - Les opérateurs

+	addition
++	incrémentation
–	soustraction
––	décrémentation
*	multiplication, pointeur
/	division
%	modulo
=	affectation
–>	référence au champ d'un enregistrement
,	séparateur d'articles dans une liste
;	séquentialité
==	égalité
!=	inégalité
<	inférieur
>	supérieur
<=	inférieur ou égal
>=	supérieur ou égal
<<	décalage gauche
>>	décalage droit
?	opérateur ternaire de condition
&	ET bit à bit
^	OU exclusif bit à bit
|	OU inclusif bit à bit
&&	ET
||	OU
( )	opérateur de caste
sizeof	taille

1 - Notions fondamentales

4 - 6 - Vocabulaire

On définit les termes suivants :

- un "type entier" désigne un entier (`int`), un entier court (`short`), un entier long (`long`) ou un caractère (`char`)
- un "type arithmétique" désigne un "type entier", un réel (`float`) ou un réel double (`double`)
- un "objet" est une zone mémoire qui peut être manipulée d'une certaine manière. Des exemples courants d'objets sont les variables ainsi que les variables structurées
- une "lvalue" désigne une valeur qui fait référence à un objet dont on peut modifier le contenu :
  une lvalue peut être :
  - une variable simple
  - un élément d'un tableau
  - un champ d'une structure
  - l'objet repéré par un pointeur

2 - Types de base

La manipulation d'une information dans un programme se fait au travers de son identificateur de variable auquel est associé un type. Le type détermine la façon dont l'information est codée en mémoire . Traditionnellement, on distingue les types simples ou types de base des types composés. Ces derniers sont caractérisés par le fait qu'ils sont constitués d'un ensemble de valeurs.

1 - Types simples

1 - 1 - Type caractère

Les objets de ce type sont les caractères appartenant au code ASCII. Les caractères sont représentés sur un octet (8 bits). La norme ANSI prévoit en fait 2 types caractère :

- un type signé
- et un type non signé :

```
[signed | unsigned] char
```

2 - Types de base

1 - 2 - Type entier

Les entiers sont représentés en complément à deux. La précision avec laquelle ils sont représentés dépend de la machine sur laquelle on travaille. Cette précision peut varier de 8 à 64 bits. En général, **`int`** fait référence à la longueur du mot machine, chaque compilateur étant libre d'interpréter **`short`** et **`long`** de manière adaptée à sa propre structure naturelle.

```
[signed | unsigned]  [long | short] int
[signed | unsigned]  {long | short} [int]
```

L'attribut **`unsigned`** déclare un objet non signé. Dans ce cas, les règles arithmétiques habituelles pour les nombres signés n'ont plus cours. Les entiers non signés sont représentés en binaire pur.

2 - Types de base

1 - 3 - Type flottant

Les types flottants appelés par abus de langage type réel servent à représenter de manière approchée une partie des nombres réels. La représentation des flottants se fait sous la forme : $(-1)^s*m*b^e$ avec pour s, m, b et e les significations suivantes :

- s le signe de la mantisse (0 positif, 1 négatif),
- m la mantisse avec bit caché dans certaines implémentations,
- b la base de numération
- e l'exposant

```
{[long]  double | float}
```

`float`, **`double`** et **`long double`** désignent respectivement des nombres en simple ou double précision. Celle-ci varie de 32 à 64 ou 80 bits. La norme ANSI impose seulement que la précision soit au minimum de 6 (entiers d'au plus 6 chiffres) pour **`float`** et 10 pour double et **`long double`**.

1 - 4 - Type void

Il ne s'agit pas d'un type à proprement parler. Il sert à déclarer une fonction ne retournant aucun résultat. Ce type représente aussi un ensemble vide de valeurs.

2 - Types de base

2 - Déclaration

La forme générale d'une déclaration est la suivante :

```
[classe_de_mémorisation] [qualifieurs] type identificateur [= valeur];
```

Le rôle d'une déclaration est d'associer un type à un identificateur et de permettre la réservation d'espace mémoire. La déclaration autorise en outre l'affectation à la variable d'une valeur initiale d'une manière souple et bien adaptée.
En l'absence d'initialisation explicite, les variables externes ainsi que les variables statiques sont systématiquement mise à 0.
Les variables automatiques et registres peuvent avoir n'importe quelle valeur, généralement une valeur parasite.

2 - 1 - Classe de mémorisation

Classe de mémorisation	Niveau de déclaration
`extern`	global ou local
`static`	global ou local
`auto`	local
`register`	local
`typedef`	global ou local (artifice syntaxique)

2 - Types de base

2 - 2 - Qualifieurs

- `const`
  Le rôle de ce qualifieur est d'indiquer que le contenu d'une variable ne doit pas changer. Cependant, quelle que soit la bonne volonté du compilateur, des modifications sont toujours possibles. Une variable ayant reçu le qualifieur `const` à sa déclaration doit être initialisée à deux exceptions près :
  - elle possède en plus le qualifieur `volatile` : elle pourra donc être modifiée indépendamment du programme. Son initialisation n'est donc pas indispensable mais possible.
  - il s'agit de la redéclaration d'une variable globale (par `extern`). L'initialisation a dû être faite ailleurs. Qui plus est l'initialisation est interdite à ce niveau.
- `volatile`
  Ce qualifieur a pour but d'indiquer au compilateur qu'une variable peut voir son contenu évoluer indépendamment des instructions du programme. Un tel changement peut être provoqué par une interruption ou par un périphérique qui agit sur des emplacements particuliers de la mémoire.
  L'intérêt de ce qualifieur est d'interdire au compilateur d'effectuer certaines optimisations.
- `restrict`
  permet des optimisations au compilateur.

Exemples :

```
const int y = 3 * 6;
static long int z;
extern float a;
volatile long double c;
```

3 - Opérateurs et expressions

1 - Les expressions

Une expression est une construction définissant des règles de calcul pour l'élaboration d'une valeur en exécutant des opérations (représentées par des opérateurs) sur des opérandes (représentés par des variables, des constantes ou des appels de fonction).
Elle peut être utilisée dans des instructions d'affectation ou dans des expressions conditionnelles, afin de contrôler l'exécution d'un programme.
Les traitements réalisés, pour les exceptions pouvant survenir lors de l'évaluation d'une expression (division par zéro, etc…), dépendent de la mise en œuvre du langage.

2 - Règles de conversion

Lorsqu'une expression est évaluée, il est parfois nécessaire que la valeur d'un opérande soit converti dans un type différent.

2 - 1 - Les caractères et les entiers

Les types `int`, `short`, `long` et `char` forment un groupe de types entiers de précisions différentes. Habituellement, une valeur de n'importe lequel de ces types peut être utilisée partout où une valeur de l'un de ces différents types est attendue.

3 - Opérateurs et expressions

La précision n'étant pas identique d'un type à l'autre, certaines conversions peuvent s'avérer nécessaires. Ces éventuelles conversions obéissent à plusieurs règles :

- Un entier est converti en un entier plus court ou en un caractère en ignorant les octets de poids fort. Par exemple, affecter la valeur 256 d'une variable `int` à une variable `char` lui donne la valeur 0.

- Un entier signé est converti en un entier plus long avec extension du bit de signe. Cela peut toutefois aboutir à des résultats étonnants lorsque la conversion se fait vers un entier non signé. Ainsi une valeur négative peut très bien donner comme résultat une très grande valeur positive. Par exemple, affecter la valeur `-1` d'une variable `int` à une variable `unsigned short` lui donne la valeur `65535`.

- Un entier non signé ou un caractère est converti en un entier plus long en complétant par des zéros.

- Les valeurs de tailles identiques ne sont pas modifiées. Cependant, des valeurs signées et non signées peuvent produire d'étranges résultats lorsque la valeur source ne peut être représentée dans le type destination.

3 - Opérateurs et expressions

2 - 2 - Les flottants

Tous les nombres flottants (`float`, `double` et `long double`) sont convertis en double précision à chaque fois qu'une valeur de type flottant apparaît dans une expression. Cette conversion s'effectue en rajoutant des zéros à la partie fractionnaire.
Quand un `double` est converti en `float`, il est arrondi avec une perte éventuelle de précision.

2 - 3 - Les flottants et les entiers

Un entier est converti en `double` sans aucun problème et en `float` sans perte de précision, à moins que cet entier ne contienne trop de chiffres que la précision d'un `float` ne permet pas de représenter.
Quand un nombre flottant est converti en `int`, il est possible que le résultat entier soit incorrect si la valeur du nombre flottant est en dehors de la limite de représentation des entiers.
Ces conversions dépendent de la machine sur laquelle elles sont effectuées.

2 - 4 - Les pointeurs et les entiers

Les entiers peuvent être additionnés ou soustraits à des pointeurs, mais ne peuvent en aucun cas leur être affectés à l'exception de la valeur 0. Deux pointeurs sur des objets de même type peuvent être soustraits.

3 - Opérateurs et expressions

2 - 5 - Les objets signés et non signés

Lorsqu'une valeur signée est convertie en une valeur non signée, ou vice-versa, il s'agit d'un simple changement de type : la chaîne de bits reste la même.

3 - Les conversions automatiques

Lorsqu'une expression est évaluée, il se produit automatiquement un certain nombre de conversions selon les règles suivantes :

- Tout opérande de type **`char`** ou **`short`** est converti en **`int`**. Ce qui signifie que si on additionne deux **`short`** pour affecter le résultat à un **`short`**, les deux opérandes sont convertis en **`int`**, l'addition se fait entre **`int`**, puis le résultat est tronqué pour être affecté à un **`short`**.
- Tout opérande de type **`float`** est converti en **`double`**, et ceci même si les deux opérandes ont le même type.
- Si l'un des opérandes est respectivement un **`double`**, un **`long`** ou un **`unsigned`**, et si l'autre est différent, il est converti dans le type du premier opérande.

3 - Opérateurs et expressions

op1/op2	char, short, int	unsigned	long	foat, double
char, short, int	int	unsigned	long	double
unsigned	unsigned	unsigned	long	double
long	long	long	long	double
float, double	double	double	double	double

3 - Opérateurs et expressions

4 - Les opérateurs

4 - 1 - Les opérateurs unaires

Les opérateurs unaires apparaissent à droite ou à gauche d'une expression et ne possèdent qu'un seul opérande.

4 - 1 - 1 Opérateurs d'accès aux objets

- `[]` indexation
  Il existe deux manières d'accéder aux éléments d'un tableau :
  - La première consiste à utiliser un index entre crochets après le nom du tableau.
  - La seconde est d'utiliser un nom de tableau comme pointeur, de telle manière que `A[i]` par exemple, soit interprété comme `*(A+i)`

- `&` adresse de
  S'applique à toute chose qui est une lvalue à l'exception des champs de bits des structures et des variables de type registre. L'opérateur réalise une évaluation d'adresse et rend une valeur de type pointeur vers chose.

  Exemple :

  soient un `int i` et un pointeur vers un `int pi,` on peut écrire

  ```
  pi = &i;
  ```

3 - Opérateurs et expressions

- `*` contenu de
  Cet opérateur est synonyme d'indirection. Il s'applique à une expression délivrant une valeur de type pointeur vers chose. Il peut toutefois être utilisé avec le nom d'un tableau pour accéder aux éléments de ce tableau.

  Exemple :

  `*pi = 2;` affecte la valeur 2 à l'entier pointé.
  `int t[100];`
  `*(t) = 0;` initialise à `0` le 1er élément du tableau.
  `*(t+1) = 0;` initialise à `0` le 2ème élément du tableau.

- `.` sélection
  Cet opérateur permet d'accéder aux différents champs dune structure ou d'une union. Il doit suivre un identificateur de structure ou d'union et doit précéder un identificateur de champ.

  Exemple :

  `struct {int pi, pr} complexe;`
  `complexe.pi;` désigne la champ `pi` de la structure `complexe`.

3 - Opérateurs et expressions

- `->` sélection
  Cet opérateur permet également d'accéder aux différents champs d'une structure ou d'une union, à ceci près qu'il constitue en quelque sorte une abréviation du premier opérateur de sélection dans la mesure où il s'agit ici de pointeurs vers des structures ou des unions. Cet opérateur est une combinaison des opérateurs `*` et `.`

  Exemple :
  ```
  struct {int pi, pr} *complexe;
  complexe->pi
  ```
  est équivalent à `(*complexe).pi`

3 - Opérateurs et expressions

4 - 1 - 2 Opérateurs arithmétiques

- `-` moins unaire
  Il génère la négation de l'expression se trouvant à sa droite, cette expression devant bien entendu être numérique. Réciproquement le `+` unaire n'existe pas.

- `++` `--` pré-incrémentation et post-décrémentation
  Ils peuvent s'appliquer à leurs opérandes, de type arithmétique ou pointeur de deux manières :
  - comme opérateurs préfixés : l'objet référencé par la lvalue est incrémenté ou décrémenté de `1`. La valeur résultant de l'expression est cette nouvelle valeur.
  - comme opérateurs postfixés : le résultat de l'expression est la valeur initiale de l'objet référencé par la lvalue. L'objet est ensuite incrémenté ou décrémenté de `1`.

  Lorsque l'un de ces deux opérateurs est appliqué à un objet de type pointeur, cet objet est incrémenté ou décrémenté de la taille du type de l'objet sur lequel il pointe.

3 - Opérateurs et expressions

Exemple 1 :

```
int t[10], i;
i = 0;
t[i++] = 0;
```
met à 0 l'élément 0 du tableau

```
t[i++] = 0;
```
met à 0 l'élément 1 du tableau

Exemple 2 :

`a = 6;`	
`b = ++a;`	`b` prend la valeur 7, `a` prend la valeur 7
`c = --a;`	`c` prend la valeur 6, `a` prend la valeur 6
`d = a++;`	`d` prend la valeur 6, `a` prend la valeur 7
`e = a--;`	`e` prend la valeur 7, `a` prend la valeur 6

3 - Opérateurs et expressions

4 - 1 - 3 Opérateurs logiques

- `!` négation logique
  Il s'applique à tous les types arithmétiques ainsi qu'aux pointeurs. Le type du résultat de l'expression est `int` ou pointeur selon l'objet auquel il est appliqué
  `!x` vaut `1` si `x` vaut `0`
  `!x` vaut `0` si `x` est différent de `0`

- `^` complément à un bit à bit
  Il s'applique à tous les objets de type entier et produit le complément à un de l'opérande.

4 - 1 - 4 Autres opérateurs unaires

- `(type)` opérateur de caste
  Cet opérateur réalise une conversion de type sur l'opérande.

  Exemple :
  soient `f` un `float` et `i` un `int`
  `f = (float) i;` convertit l'entier en un flottant.

3 - Opérateurs et expressions

- `sizeof()` opérateur de taille
  Cet opérateur s'utilise de deux manières :

```
sizeof(expression)
sizeof(type)
```

Dans le premier cas, il permet dévaluer la taille de l'expression qui suit.

- Lorsque cet opérateur s'applique à un tableau ou à une structure, le résultat est le nombre total d'octets contenu dans le tableau ou la structure.
- Lorsque cet opérateur s'applique à une union, le résultat est la plus grande taille de l'union.

Dans le deuxième cas, il fournit la taille de l'objet désigné

Exemple :

```
int t[10];
sizeof(int);  retourne comme résultat 4
sizeof(t);    retourne comme résultat 40
```

3 - Opérateurs et expressions

4 - 2 - Les opérateurs binaires

4 - 2 - 1 Opérateurs multiplicatifs

- `*` multiplication

- `/` division
  Si les deux opérandes sont entiers, la division est une division entière. Si l'un des deux opérandes est flottant, l'autre opérande sera converti et la division donnera un résultat flottant.
  En ce qui concerne la division entière, si les deux opérandes sont positifs, l'arrondi se fera vers les valeurs inférieures. Si l'un des deux opérandes est négatif, la façon dont se fera l'arrondi dépend de la machine.

- `%` modulo
  Cet opérateur s'applique à des opérandes de type entier et délivre le reste de la division entière.

4 - 2 - 2 Opérateurs additifs

- `+` `–` addition et soustraction
  Ces deux opérateurs s'appliquent aux valeurs de type arithmétique de manière classique. Ils peuvent aussi servir à additionner ou soustraire un entier à un pointeur. Dans ce cas, la valeur entière est multipliée par la taille en octets de l'objet pointé.
  Enfin, il est possible de soustraire deux pointeurs de même type. Le résultat de la soustraction est un entier représentant la différence entre les deux pointeurs, divisée par la taille en octets de l'objet pointé.

  Exemple :

```
int t[10];
p = &t[0];
p = p + 2;
```

`p` repère l'élément d'indice `2`

4 - 2 - 3 Opérateurs de décalage

- `<<` décalage gauche

- `>>` décalage droit
  Les deux opérandes doivent être de type entier. L'opérande gauche de ces deux opérateurs délivre l'objet à décaler, l'opérande droite délivre le nombre de bits sur lequel porte le décalage.
  Les deux opérandes doivent être de type entier. Le résultat n'est pas défini si l'opérande droit est négatif ou si il est supérieur à la taille en bits de l'objet à décaler. Le résultat est du type de l'opérande gauche.
  `<<` est toujours un décalage logique.
  `>>` est un décalage logique si l'opérande gauche est non signé, sinon il peut être logique ou arithmétique, dépendant de la machine.
  Lors d'un décalage logique, les bits en entrée sont égaux à 0, tandis que lors d'un décalage arithmétique, ils prennent la valeur du bit de signe.

  Exemple :
  `i = 2;`
  `i = i << 3;` `i` prend la valeur `16`
  `i = 35 >> 3;` `i` prend la valeur `4`

3 - Opérateurs et expressions

4 - 2 - 4 Opérateurs relationnels

Ils s'appliquent aux valeurs de type arithmétique, ou à deux pointeurs. Ils sont les suivants :

- < inférieur
- > supérieur
- <= inférieur ou égal
- >= supérieur ou égal
- == égalité
- != différence

4 - 2 - 5 Opérateurs logiques

- `&&` et logique
- `||` ou logique

Ces deux opérateurs s'appliquent à des valeurs booléennes en prenant les conventions suivantes :

vrai : n'importe quelle valeur différente de `0`
faux : `0`

L'évaluation des expressions s'effectue de la gauche vers la droite et s'arrête dès que le résultat de l'évaluation est connu.
Si `a` est vrai et `b` faux, dans l'expression `a && b && c, c` ne sera pas évalué.

Exemple :

```
(p == null) || (p->size < 10)
```

Dans cet exemple, il est crucial que le second opérande ne soit pas évalué si le premier est vrai.

3 - Opérateurs et expressions

4 - 2 - 6 Opérateurs de bits

- `&` et bit à bit

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

- `^` ou exclusif bit à bit

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

- `|` ou inclusif bit à bit

a	b	a & b
0	0	0
0	1	1
1	0	1
1	1	1

Ces opérateurs s'appliquent aux valeurs de type entier. Ils fonctionnent sur les bits de leurs opérandes.

4 - 3 - Opérateur ternaire

- `?` expression conditionnelle
  Il s'agit donc d'un opérateur ternaire s'utilisant de la façon suivante :

  ```
  expression_1 ? expression_2 : expression_3;
  ```

  `expression_1` est évaluée.
  Si sa valeur est différente de `0`, l'opérateur `?` retourne la valeur `expression_2`, sinon il retourne la valeur `expression_3`

  Cet opérateur a une associativité qui va de la vers la droite vers la gauche, ce qui implique que l'expression

  ```
  x ? y : z ? a : b
  ```

  signifie

  `x ? y : (z ? a : b)` , et non pas `(x ? y : z) ? (a : b)`

Exemple :

```
i = 5;
x = (i % 2) ? 3 : 4;
```

x prend la valeur `3`

3 - Opérateurs et expressions

4 - 4 - Opérateur d'affectation

L'opérateur d'affectation est le symbole =. Il s'utilise de la manière suivante :

```
lvalue = expression;
```

L'expression est évaluée et affectée à la lvalue après une éventuelle conversion. Cet opérateur ne se contente pas d'affecter une valeur à une variable, il produit un effet de bord et délivre comme résultat la valeur affectée. Son associativité va de la gauche vers la droite.

Exemple :

```
a = b = c = d;  a la signification suivante : a = (b = (c = d));
```

Il est aussi possible de combiner un des opérateurs binaires suivants à une affectation :

```
+= -= *= /= %= >>= <<= &= ^= |=
```

Tous ces opérateurs s'utilisent de la même manière : `lvalue operateur = expression;`

dont la sémantique est : `lvalue = lvalue operateur expression;`

mais ou `lvalue` n'est, bien sûr, évaluée qu'une seule fois.

3 - Opérateurs et expressions

4 - 5 - Opérateur `,`

Cet opérateur s'utilise pour combiner deux expressions en une seule. Sa forme est la suivante :

`expression_1, expression_2`

`expression_1` et **`expression_2`** sont évaluées et l'opérateur '`,`' rend **`expression_2`**

`expression_1` n'intervenant pas dans le résultat, cela n'a de sens que si **`expression_1`** réalise un effet de bord

Exemple :

`i = (j = 1, j + 2);` donne à `i` la valeur `3`

3 - Opérateurs et expressions

4 - 6 - Priorité des différents opérateurs

Ces opérateurs sont présentés par ordre de priorité décroissante.

Priorité	Opérateurs	Associativité
15	`() [] -> .`	->
14	`! ~ ++ -- - (type)` *(indirection) &(adresse de) `sizeof`	<-
13	*(multiplication) `/ %`	->
12	`+ -`	->
11	`<< >>`	->
10	`< <= > >=`	->
9	`== !`	->
8	&(sur chaîne de bits)	->
7	^(ou exclusif sur chaîne de bits)	->
6	|(ou inclusif sur chaîne de bits)	->
5	`&&`	->
4	`||`	->
3	`?`	<-
2	`= += -= *= /= %= >>= <<= &= ^= |`	<-
1	`,`	->

4 - Instructions

1 - Instructions simples et composées

1 - 1 - Instruction simple

Une expression devient une instruction lorsqu'elle est suivie d'un point-virgule (`;`). Une instruction simple a donc la forme suivante :

```
expression;
```

1 - 2 - Instruction composée

Une instruction composée, connue aussi sous le nom de bloc, est une suite d'instructions. Elle commence par l'accolade ouvrante `{` et se termine par l'accolade fermante `}`.
Elle a la forme suivante :

```
{
  liste_d_instructions;
}
```

Elle peut remplacer n'importe quelle instruction simple. De nouvelles variables peuvent être déclarées au début de n'importe quelle instruction composée

4 - Instructions

2 - Instructions de choix

2 - 1 - Instruction conditionnelle

```
if (expression) instruction;
if (expression)
    instruction_1;
else
    instruction_2;
```

2 - 1 - Instruction switch

```
switch (expression) {
   case constante_scalaire_1 : liste_d_instructions_1;
   case constante_scalaire_2 : liste_d_instructions_2;
   etc …
   default : liste_d_instructions;
}
```

4 - Instructions

`expression` est évaluée. Elle doit rendre un résultat de type entier. Cette expression est comparée aux constantes derrière les **`case`**. A la première correspondance des deux valeurs, la liste d'instructions correspondante est exécutée.

Si aucune constante n'est égale à expression :

- s'il existe une partie **`default`**, elle est exécutée
- sinon, l'instruction **`switch`** ne fait rien.

Attention

1. Une fois terminée l'exécution d'une liste d'instructions d'un case, on continue en séquence dans le case suivant. Si on ne veut pas que ce genre de phénomène se produise, il faut utiliser l'instruction **`break`**

2. la syntaxe du **`switch`** est en fait :
   `switch (expression) instruction_composée`

Exemple :

```
switch (dpt) {
        case 22 :   printf("Côtes d'Armor\n"); break;
        case 29 :   printf("Penn ar bed\n"); break;
        case 35 :   printf("Ille et Vilaine\n"); break;
        case 56 :   printf("Morbihan\n"); break;
        default :   printf("Département inconnu !\n");
}
```

4 - Instructions

3 - Instructions de contrôle d'itération

3 - 1 - Instruction `while`

```
while (expression) instruction;
```

Cette boucle sera effectuée zéro ou plusieurs fois.

3 - 2 - Instruction `do`

```
do instruction; while (expression);
```

Cette suite d'instructions sera exécutée au moins une fois.

4 - Instructions

3 - 2 - Instruction `for`

```
for (expression_1; expression_2; expression_3) instruction;
```

Cette instruction est une forme singulière et pratique de l'instruction `while`. Elle a la forme suivante :

```
expression_1;
while (expression_2) {
    instruction;
    expression_3;
}
```

Exemples :

```
int x = 10;
while (x) {
    x--;
}

do {
    x++;
} while (x < 10);

int somme = 0, i;
for (i = 0; i < 10;  i++ {
    somme += i;
}
```

4 - Instructions

4 - Instructions de rupture de séquence

4 - 1 - Instruction `break`

Cette instruction fournit un mécanisme pour arrêter une boucle prématurément. Elle permet aussi de sortir d'un cas de l'instruction `switch`, ce qui est nécessaire pour éviter d'exécuter en séquence le reste des instructions composant le `switch`.

4 - 2 - Instruction `continue`

Cette instruction est très proche de l'instruction break mais ne s'applique pas à l'instruction `switch`. Elle ne s'applique en effet qu'aux instructions permettant le contrôle d'itération.
Elle provoque l'arrêt de l'itération courante et le passage au début de l'itération suivante dans une boucle contrôlée par une instruction `while`, `do` ou `for`.
Dans une boucle for, la partie réinitialisation de la boucle (`expression_3`) est exécutée.

4 - 2 - Instruction `return`

Cette instruction provoque le retour chez l'appelant de la fonction courante. Les deux formes de l'instruction `return` sont :

```
return;
return expression;
```

5 - Bibliothèque d'Entrée/Sortie standard (1/2)

1 - Généralités

Les Entrées/Sorties ne constituent pas une partie intrinsèque du langage. Pour utiliser la bibliothèque d'E/S standard du C, il faut inclure dans le programme source le contenu du fichier **`stdio.h`** (option **`-lc`** à la compilation). Ce fichier contient un certain nombre de définitions qui sont nécessaires pour utiliser les E/S standards.
L'inclusion de ce fichier dans le programme source se fait à l'aide de la directive :

```
#include <stdio.h>
```

Tout programme C possède 3 voies standards de communication qui sont :

- **`stdin`** généralement le clavier
- **`stdout`** généralement l'écran
- **`stderr`** généralement l'écran (sortie des erreurs)

5 - Bibliothèque d'Entrée/Sortie standard (1/2)

2 - E/S caractères

2 - 1 - Entrées caractères

`int getchar()`

`getchar` est une macro qui retourne le prochain caractère de l'entrée standard. Cette macro est l'équivalent de la fonction : **`fgetc(stdin)`**

`char *gets(char *ch)`

`gets` est une fonction permettant de lire une chaîne de caractères qu'elle place dans **`ch`**. Elle retourne en résultat l'adresse de la chaîne saisie ou **`NULL`** en cas d'erreur.

2 - 2 - Sorties caractères

`int putchar(int c)`

`putchar` est aussi une macro. **`putchar(c)`** envoie sur la sortie standard le caractère passé en paramètre. Cette macro est l'équivalent de la fonction : **`fputc(c, stdout)`**

`int puts(const char *ch)`

`puts` est une fonction permettant d'écrire une chaîne de caractères suivie d'une fin de ligne **`\n`**. Elle retourne **`NULL`** en cas d'erreur ou une valeur non négative sinon.

5 - Bibliothèque d'Entrée/Sortie standard (1/2)

3 - E/S formatées

3 - 1 - Entrées formatées

Les entrées s'effectuent à l'aide de **`scanf`**, et **`sscanf`**.

```
int scanf(const char *format, arg1, …, argn)
```

La fonction **`scanf`** admet un nombre variable de paramètres. Les différents arguments sont tous des pointeurs. **`scanf`** lit l'entrée standard en l'interprétant selon le format passé en premier argument. Le résultat de cette interprétation a pour effet de mettre des valeurs dans les variables pointées par les différents **`arg`**$_i$. Elle retourne le nombre de caractères lus ou la valeur **`EOF`** en cas d'erreur.

Une séquence d'interprétation (format) a la forme suivante :

```
%[*][nombre][modificateur]caractère
```

5 - Bibliothèque d'Entrée/Sortie standard (1/2)

- `*` la présence de ce symbole indique que la prochaine unité du flot d'entrée doit être ignorée,
- `nombre` indique la longueur maximum de l'unité à reconnaître.
- `caractère` indique comment sera interprétée la prochaine unité. Il peut prendre l'une des valeurs suivantes :
  - `d` nombre entier
  - `o` nombre octal
  - `x` nombre décimal
  - `u` entier non signé
  - `c` caractère
  - `s` chaîne de caractères
  - `f` nombre flottant
  - `p` pointeur
- `modificateur` ce paramètre n'a de signification que pour les entiers et les réels :
  - `h` entier court
  - `l` entier long ou réel double (long double)
  - `L` réel double (double)

```
int sscanf(char *zone, const char *format, arg1, …, argn)
```

`sscanf` a un comportement similaire à `scanf` sauf que cette fonction ne lit pas l'entrée standard mais dans une chaîne de caractère zone passée en paramètre.

5 - Bibliothèque d'Entrée/Sortie standard (1/2)

3 - 2 - Sorties formatées

Les sorties formatées s'effectuent à l'aide de **`printf`**, et **`sprintf`**.

```
int printf(const char *format, arg1, …, argn)
```

La fonction **`printf`** admet un nombre variable de paramètres. Elle convertit les valeurs des arguments en fonction du format spécifié, puis écrit le résultat sur la sortie standard. Elle retourne le nombre de caractères écrits ou une valeur négative en cas d'erreur.
Une séquence d'interprétation introduite par le caractère **`%`** a la forme suivante :
`%[+][-][nombre1][.nombre2][modificateur]caractère`

- **`+`** — le signe plus apparaît toujours au début,
- **`-`** — l'argument est cadré à gauche dans son champ d'impression,
- **`nombre1`** — indique la taille minimum du champ d'impression,
- **`nombre2`** — nombre maximum de caractères à imprimer si **`arg`**$_i$ est une chaîne,
  nombre de chiffres à imprimer après le point décimal si **`arg`**$_i$ est un flottant
- **`caractère`** — identique à **`scanf`**
- **`modificateur`** — identique à **`scanf`**

```
int sprintf(char *zone, const char *format, arg1, …, argn)
```

`sprintf` a un comportement similaire à **`printf`** sauf que cette fonction n'écrit pas sur la sortie standard mais dans une zone mémoire passée en paramètre.

5 - Bibliothèque d'Entrée/Sortie standard (1/2)

3 - 3 - Formats utilisables : synthèse

Type de l'expression	Code
`char` `unsigned char`	`c`
`short int`	`hd, hi`
`unsigned short int`	`hu, ho, hx, hX`
`int`	`d, i`
`unsigned int`	`u, o, x, X`
`long int`	`ld, li`
`unsigned long int`	`lu, lo, lx, lX`
`float`	`f, e, E, g, G`
`double`	`Lf, Le, LE, Lg, LG`
`long double`	`lf, le, lE, lg, lG`
`void *`	`p`

6 - Types agrégés

1 - Introduction

Dans de très nombreuses situations, les types de base s'avèrent insuffisants pour permettre de traiter un problème.
Le langage C comporte la notion de type agrégé :

- tableau,
- structure,
- union,
- énumération.

2 - Les tableaux

Un tableau est un ensemble d'éléments de même type désignés par un identificateur unique. Chaque élément est repéré par une valeur entière appelée indice indiquant sa position dans l'ensemble.
Les tableaux sont toujours à bornes statiques et leur indiçage démarre toujours à partir de `0`.

`type identificateur[dimension`$_1$`]…[dimension`$_n$`]`

6 - Types agrégés

Exemple :

```
int t[10],                                  // tableau de 10 éléments
    m[2][5] = {2,  6, -4, 8, 11,            // tableau à 2 dimensions
               3, -1,  0, 9,  2},           // 2 lignes, 5 colonnes
    x[5][12][7],
float f[20];
…
t[2] = 6;
printf("%d", m[1][3]); // affiche la valeur 9
```

L'identificateur du tableau désigne non pas le tableau dans son ensemble, mais plus précisément l'adresse en mémoire du début du tableau. Ceci implique :

- qu'un tableau ne pourra jamais être passé en paramètre fixe à une procédure, mais toujours en paramètre modifiable et,
- qu'il est impossible d'affecter un tableau à un autre :

```
int a[10],
    b[10];
    a = b            // cette affectation est interdite
```

6 - Types agrégés

3 - Structure

Une structure est un objet agrégé comprenant un ou plusieurs champs (membres) d'éventuellement différents types que l'on regroupe sous un seul nom afin d'en faciliter la manipulation et le traitement. Chacun des champs peut avoir n'importe quel type, y compris une structure, à l'exception de celle à laquelle il appartient.
Les structures peuvent être imbriquées les unes dans les autres. Dans le cas d'une structure se référant à elle-même, l'interdiction précédente peut être contournée en utilisant un champ qui est un pointeur vers cette même structure. Cette structure doit alors impérativement porter un nom (étiquette).

```
struct [etiquette] {
   type champ1;
   …
   type champn;
} [identificateur];
```

Exemple :

```
struct date {
    int jour,
        mois,
        année;
} x, y = {17, 11, 2003};  // initialisation de la structure y
```

6 - Types agrégés

`date` est le nom de la structure, tandis que `x`, et `y` sont deux variables.

Pour définir une troisième variable `z` il suffit d'écrire :

```
struct date z;
```

La taille d'une structure est la somme des tailles de tous les objets qui la compose.

3 - 1 - Accès aux champs dune structure

Il faut distinguer 2 cas :

- la structure est délivrée par une variable :
  Cet accès se fait à l'aide de l'opérateur `.` (point)

  Exemple :

  `x.mois` désigne le champ `mois` de la variable `x`

- la structure est délivrée par un pointeur `p` :
  Cet accès se fait à l'aide de l'opérateur `->` (indirection)

  Exemple :

  `p->jour` désigne le champ `jour` de la variable `p`
  `(*p).jour`

6 - Types agrégés

3 - 2 - Champs de bits

Quand l'espace mémoire est limité, il peut s'avérer intéressant de rassembler plusieurs objets dans un seul mot machine. Ainsi peut-on définir des champs de bits à l'intérieur d'un entier à l'aide d'une définition du même genre que celle d'une structure.

Exemple :

```
struct etat {
          unsigned sexe         : 1;   // masculin, féminin
          unsigned situation    : 3;   // célibataire, marié etc...
          unsigned exempte      : 1;   // oui, non
} e = {0, 7, 1};
```

Les expressions derrière le symbole `:` (deux points) doivent délivrer une constante. Elles indiquent la longueur du champ, exprimée en bits.

Le nom d'un champ de bits peut être omis lorsqu'on ne s'y intéresse pas.
Une longueur de bits de `0` est une convention pour demander au compilateur d'aligner le prochain champ sur une frontière d'entier.

6 - Types agrégés

4 - Union

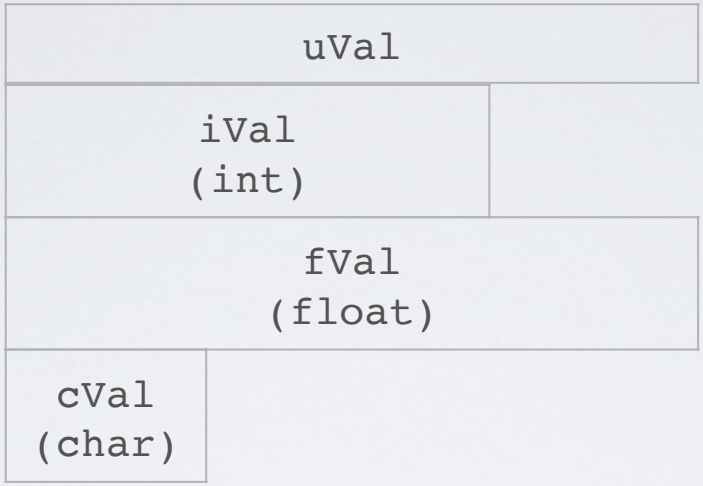
Une union est conceptuellement identique à une structure mais peut, à tout moment, contenir n'importe lequel des différents champs. Une structure définit une suite de champs, portant chacun un unique nom et tous présents simultanément. Ils sont rangés dans des emplacements mémoire consécutifs. Une union, d'un autre côté, définit plusieurs manières de regarder le même emplacement mémoire. A l'exception de ceci, la façon dont sont déclarées et référencées les structures et les unions est identique.

```
union [etiquette] {
   type champ1;
   …
   type champn;
} [identificateur];
```

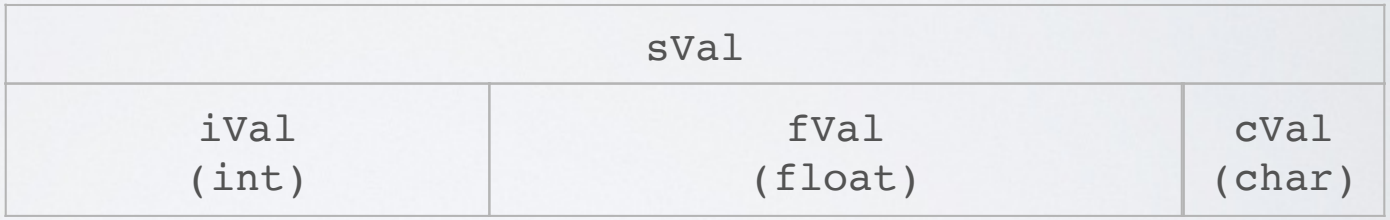
6 - Types agrégés

Exemple :

```
union {
    int   iVal;
    float fVal;
    char  cVal;
} uVal;
```




```
struct {
    int   iVal;
    float fVal;
    char  cVal;
} sVal;
```



Une telle définition définit une variable de nom **`uval`** et de type union qui peut, à tout moment, contenir soit un entier, soit un réel, soit un caractère.
La taille mémoire de cette variable **`uval`** est égale à la taille mémoire du plus grand type qu'elle contient. Ici c'est **`float`**.

6 - Types agrégés

5 - Enumération

Une énumération est un cas particulier de type entier. Une énumération permet à des identificateurs d'être utilisés comme des valeurs. Chacun des membres d'un type énuméré doit être explicitement déclaré.

```
enum [etiquette] {
   val1, …, valn
} [identificateur];
```

Exemple :

```
enum {lundi,mardi,mercredi,jeudi,vendredi,samedi,dimanche} jour;
enum {arret, marche, rapide, lent, intermittent} etat;
enum {noir, blanc, rouge=10, vert, bleu, jaune=20} couleur;
enum {false, true} b;
…
jour = vendredi;   // est identique à jour = 4;
etat = marche;     // est identique à etat = 1;
couleur = blanc    // est identique à couleur = 1;
couleur = vert     // est identique à couleur = 11;
couleur = jaune    // est identique à couleur = 20;
b = true           // est identique à b = 1;
```

7 - Fonctions

1 - Définition de fonction

```
type identificateur (paramètres_formels) {
    …
    return résultat;
}
```

2 - Passage de paramètres

Lors d'un appel de fonction, les paramètres réels doivent correspondre aux paramètres formels par leur nombre et par la compatibilité de leurs types. En C, il n'existe qu'un seul mode de passage des paramètres : c'est le passage par valeur. Il faut donc considérer un paramètre comme une variable locale ayant été initialisée avant l'appel de la fonction.
L'exception à cette règle est le cas des tableaux. En effet, lorsque le nom d'un tableau constitue l'argument d'une fonction, c'est l'adresse du premier élément qui est transmise. Ses éléments ne sont donc pas recopiés. Ce qui est le cas des chaînes de caractères aussi.
Ceci reste en effet cohérent avec le fait qu'il n'existe pas de variable désignant un tableau comme un tout. Quand on déclare `int t[10]`, `t` ne désigne pas l'ensemble du tableau. `t` est une constante de type pointeur vers un `int` dont la valeur est `&t[0]` (adresse du premier élément du tableau).

En conséquence, pour réaliser l'effet de passage de paramètre par adresse, il suffit alors de passer en paramètre un pointeur vers la variable.

7 - Fonctions

3 - Variables globales

Il n'existe pas, dans le langage C, contrairement à certains autres langages, de hiérarchisation des variables globales :

- toutes les fonctions sont au même niveau
- il n'existe qu'un seul niveau de variables globales et elles sont accessibles à tous les modules de tous les fichiers sources, sauf si l'on a volontairement limité leurs portées.

4 - Structure de bloc

Bien que le langage C ne permette pas l'imbrication de fonctions, la possibilité d'imbriquer des instructions composées existe néanmoins. Celles-ci prennent la forme d'une structure de bloc. Un bloc est délimité par une accolade ouvrante `{` et par une accolade fermante `}`. Il peut contenir des déclarations suivant immédiatement l'accolade gauche. Ce schéma peut se reproduire indéfiniment.
Si des variables sont déclarées à l'intérieur d'un bloc, la portée de telles variables est celle du bloc. Toute variable de même nom déjà déclarée dans un bloc englobant devient inaccessible dans le bloc inclus.

7 - Fonctions

5 - La fonction principale : `main`

Un programme C complet doit posséder une fonction dont le nom est **`main()`**.
Cette fonction principale est l'endroit où l'exécution du programme démarre. La déclaration de cette fonction est la suivante :

```
int main ()
{
}
```

Normalement, l'exécution du programme se termine par la sortie de la fonction **`main()`**.
Si un programme C désire accéder aux arguments de la ligne de commande, cette ligne de commande est disponible au travers de deux variables :

- **`argc`**
  la variable **`argc`** désigne le nombre total d'arguments passés dans la ligne de commande

- **`argv`**
  la variable **`argv`** est un tableau de pointeurs sur des chaînes de caractères représentant les arguments de la ligne de commande.

Ces deux variables doivent être spécifiées comme arguments formels à la fonction principale.

7 - Fonctions

Exemple : fichier `demonstration.c`

```
int main (int argc, char *argv[])
{
}
```

Exécution du fichier :

```
./demonstration 56 bruno
```

Les arguments passés à main() seront :

- `argc` = 3
- `argv[3]` = un tableau de pointeurs dont
  - le premier élément repère la chaîne "`demonstration`",
  - le second la chaîne "`56`" et
  - le troisième la chaîne "`bruno`".

7 - Fonctions

6 - Schéma de compilation

Organisation d'une unité de compilation :

- **`module.h`**
  il contient la déclaration des objets publics et la définition dans le cas des variables. Il doit être inclus dans tout fichier où les objets publics du module sont utilisés.

- **`module.c`**
  il inclut **`module.h`** et contient la définition des objets privés (statiques) et la définition des fonctions, privées ou publiques.

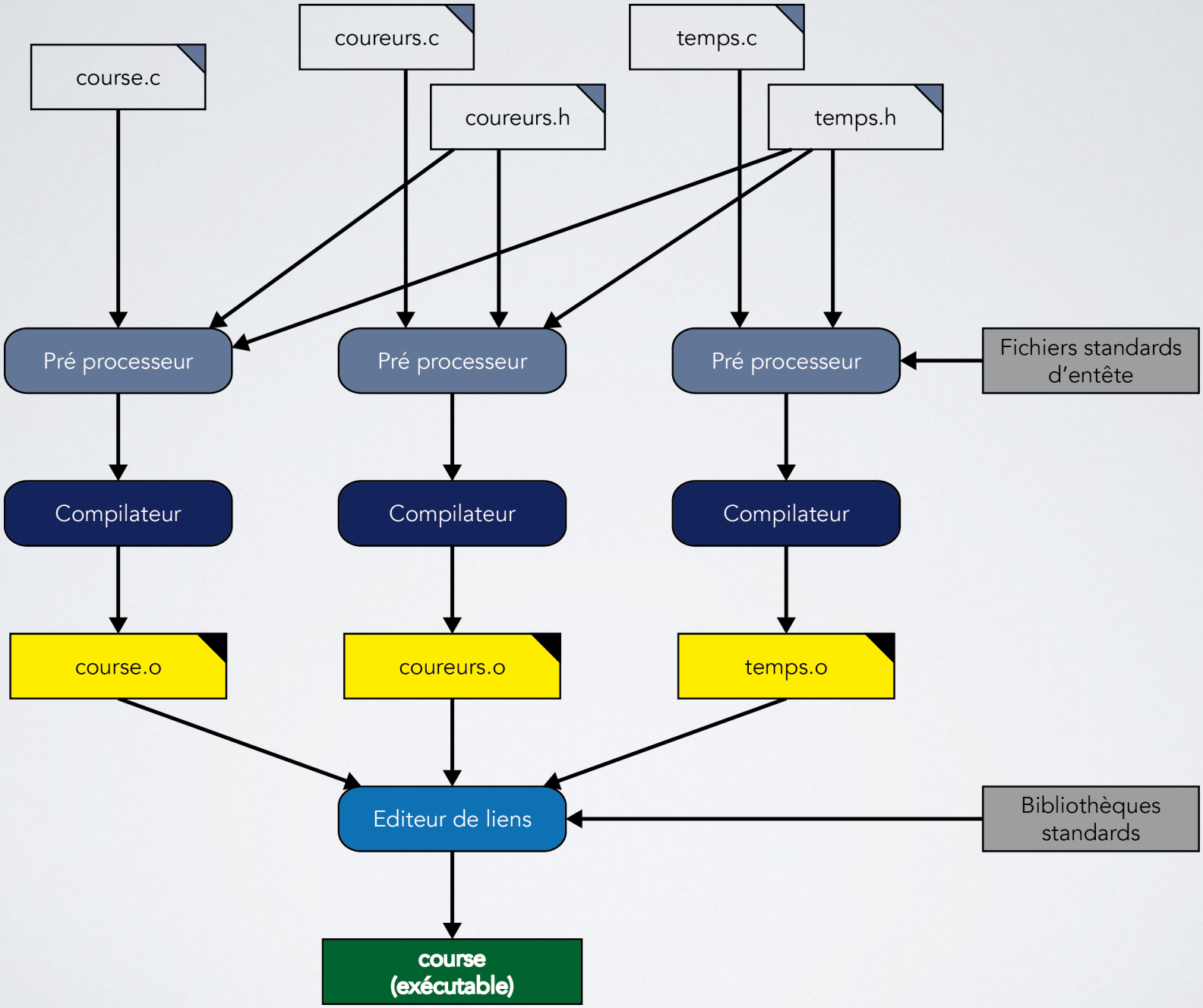
La production d'un exécutable à partir de plusieurs fichiers sources se déroule en plusieurs étapes :

1. prétraitement par le préprocesseur des fichiers sources, fondé sur l'interprétation d'instructions particulières appelées directives,

2. compilation des fichiers sources résultant du prétraitement et génération de fichiers objets non exécutables

3. édition de liens, qui a pour but de résoudre les références externes, et génération d'un exécutable.

7 - Fonctions


course.c
coureurs.c
coureurs.h
temps.c
temps.h
Pré processeur
Pré processeur
Pré processeur
Fichiers standards d'entête
Compilateur
Compilateur
Compilateur
course.o
coureurs.o
temps.o
Editeur de liens
Bibliothèques standards
course (exécutable)

8 - Bibliothèque d'Entrée/Sortie standard (2/2)

4 - E/S fichiers

La notion d'enregistrement n'existant pas en C, un fichier est vu comme une suite d'octets. Ceci a pour incidence qu'il n'existe qu'une seule organisation de fichier : l'organisation séquentielle. L'accès direct reste cependant possible dans la mesure ou le C permet un positionnement dans un fichier avec une précision d'un octet.

4 - 1 - Fonctions d'accès aux fichiers

L'association entre nom logique et nom physique s'effectue à l'ouverture du fichier. Le nom logique du fichier est en fait un pointeur sur une structure de type `FILE`.
Il existe 2 façons de considérer le contenu d'un fichier : sous forme binaire (data) ou en texte (text). Alors que certains langages distinguent clairement les 2 types de fichiers, les choses sont beaucoup plus floues en C.
En fait, la norme laisse l'implémentation libre de distinguer ou non les 2 sortes de fichiers. On parlera ainsi, lorsque l'implémentation prévoit la distinction, non plus de fichiers binaires ou texte, mais de mode d'accès binaire ou texte.

8 - Bibliothèque d'Entrée/Sortie standard (2/2)

`FILE *fopen(const char *fichier, const char *mode)`

Cette fonction permet d'ouvrir un fichier physique fichier. Elle renvoie un pointeur (fichier logique) vers le fichier ouvert et `NULL` en cas d'échec. `mode` détermine le mode d'ouverture du fichier :

- `r` lecture (le fichier doit exister)
- `w` écriture (le fichier s'il existe est écrasé)
- `a` ajout (le fichier peut exister)
- `+` mise a jour (lecture et écriture)
- `b` ouverture en mode binaire (texte par défaut)

`FILE *freopen(const char *fichier, const char *mode, FILE *flux)`

Cette fonction ferme le fichier logique flux et le rouvre en l'associant au fichier physique fichier. Cette fonction est surtout utilisée pour connecter un fichier particulier à l'un des flux prédéfinis `stdin`, `stdout` ou `stderr`.

`int fclose(FILE *flux)`

Cette fonction permet de fermer un fichier logique flux. Elle renvoie la valeur `0` si tout se passe bien et `EOF` sinon.

`void setbuf(FILE *flux, char *tampon)`

Cette fonction, si le paramètre tampon est `NULL`, supprime l'utilisation d'un tampon pour le flux concerné. Dans le cas contraire, elle permet de forcer le système à utiliser un tampon d'adresse tampon. L'appel de cette fonction est équivalent à :

```
setvbuf(flux, tampon, _IOFBF, BUFSIZ);
```

8 - Bibliothèque d'Entrée/Sortie standard (2/2)

4 - 2 - Fonctions d'E/S

4 - 2 - 1 - E/S caractères

`int fgetc(FILE *flux)`
Cette fonction lit un caractère du flux indiqué.

`char *fgets(char *ch, int n, FILE *flux)`
Cette fonction lit une chaîne de caractère du flux indiqué.

`int fputc(int c, FILE *flux)`
Cette fonction écrit un caractère sur le flux indiqué.

`int fputs(const char *ch, FILE *flux)`
Cette fonction écrit une chaîne de caractère sur le flux indiqué.

`int getline(char **buffer, size_t *tailleBuffer, FILE *flux);`
Cette fonction est identique à `fgets` à ceci près qu'elle alloue dynamiquement l'espace mémoire nécessaire (`*buffer=NULL` et `*taillebuffer=0`).
Elle réalloue l'espace mémoire s'il s'avère insuffisant.

8 - Bibliothèque d'Entrée/Sortie standard (2/2)

4 - 2 - 2 - E/S formatées

`int fscanf(FILE *flux, const char *format, arg1, …, argn)`

Cette fonction est identique à **`scanf`**, mais substitue à **`stdin`** le flux indiqué.

`int fprintf(FILE *flux, const char *format, arg1, …, argn)`

Cette fonction est identique à **`printf`**, mais substitue à **`stdout`** le flux indiqué.

4 - 2 - 3 - E/S objets

La notion d'enregistrement n'existant pas en C, elle peut néanmoins être mise en œuvre à l'aide des fonctions **`fread`** et **`fwrite`**.

`size_t fread(void *zone, size_t n, size_t nb, FILE *flux)`

Cette fonction lit, sur le flux indiqué, au maximum **`nb`** blocs (enregistrement) de **`n`** octets chacun et les range en mémoire à partir de l'adresse **`zone`**. Elle retourne le nombre de blocs effectivement lus.

`size_t fwrite(const void *zone, size_t n, size_t nb, FILE *flux)`

Cette fonction écrit, sur le flux indiqué **`nb`** blocs (enregistrement) de **`n`** octets à partir de l'adresse **`zone`**. Elle retourne le nombre de blocs effectivement écrits.

8 - Bibliothèque d'Entrée/Sortie standard (2/2)

4 - 3 - Fonctions de positionnement

`int fgetpos(FILE *flux, fpos_t *position)`

Cette fonction range à l'adresse **`position`**, la valeur courante du pointeur de fichier relatif au **`flux`** spécifié. Elle retourne **`0`** en cas de succès et une valeur négative en cas d'échec.

`int fsetpos(FILE *flux, const fpos_t *position)`

Cette fonction place le pointeur de fichier **`flux`** à l'adresse définie par **`position`**. Elle retourne **`0`** en cas de succès et une valeur négative en cas d'échec.

`int fseek(FILE *flux, long deplacement, int origine)`

Cette fonction positionne le pointeur de fichier flux a l'endroit indiqué par **`deplacement`** par rapport à **`origine`**. Le déplacement est exprimé en octets et origine peut avoir l'une des valeurs suivantes :

- **`SEEK_SET`** correspond au début du fichier
- **`SEEK_CUR`** correspond à la position actuelle du pointeur de fichier
- **`SEEK_END`** correspond à la fin du fichier

`fseek` retourne **`0`** en cas de succès et une valeur négative en cas d'échec.

`long ftell(FILE *flux)`

Cette fonction renvoie la position courante, en octets, du flux indiqué et **`-1`** en cas d'erreur.

`void rewind(FILE *flux)`

Cette fonction place le pointeur de fichier **`flux`** à son début.

8 - Bibliothèque d'Entrée/Sortie standard (2/2)

4 - 4 - Fonctions diverses

`int fileno(FILE *flux)`

Cette fonction renvoie un entier représentant le numéro d'entrée associé à ce **`flux`** dans la table des descripteurs de fichiers.

`int feof(FILE *flux)`

Cette fonction teste une fin de fichier. Elle ne peut s'appliquer qu'à un fichier déjà ouvert. Elle renvoie une valeur différente de zéro (pas obligatoirement **`EOF`**) si une fin de fichier s'est produite et 0 sinon. Son résultat n'est valide que si le fichier a été ouvert en lecture et que la dernière opération effectuée sur ce fichier était une lecture.

`int ferror(FILE *flux)`

Cette fonction teste une condition d'erreur sur un fichier ouvert. Elle renvoie une valeur différente de zéro si une erreur s'est produite sur le fichier et **`0`** sinon.

`int perror(const char *ch)`

Cette fonction écrit sur la sortie erreur standard (**`stderr`**) le contenu de la chaîne de caractère **`ch`** suivi de **`": "`** et d'un message relatif à l'erreur dont le numéro figure présentement dans la variable **`errno`**. Ce message est le même que celui obtenu par un appel de la fonction **`strerror`** avec comme argument **`errno`**.

`void clearerr(FILE *flux)`

Cette fonction réinitialise les indicateurs d'erreur et de fin de fichier du **`flux`** indiqué.

9 - Gestion mémoire

1 - Pointeurs

Les emplacements mémoire des variables utilisées dans un programme C peuvent être alloués de différentes façons :

- la mémoire des variables globales ou de celles déclarées statiques est allouée une fois pour toutes lors du chargement du programme,
- les variables locales ainsi que les arguments de fonctions ont leurs emplacements réservés dans la pile,
- l'espace mémoire des variables dynamiques est alloué explicitement par l'intermédiaire de routines de gestion mémoire.

L'emploi de variables dynamiques, manipulées par le biais de pointeurs trouve sa justification dans les situations suivantes :

- lorsqu'on ne connaît pas la taille des variables lors de la compilation, ce qui est le cas d'une table dont on ne connaît pas à priori le nombre d'éléments qu'elle est susceptible de contenir,
- lorsqu'une variable locale de taille importante est utilisée dans une fonction pouvant être appelée récursivement, risquant de ce fait un débordement de pile,
- lorsqu'on désire gérer la mémoire avec précision en adaptant dynamiquement sa taille aux besoins et en réallouant des zones précédemment libérées.

9 - Gestion mémoire

1 - 1 - Déclaration

Un pointeur est une variable dont la valeur désigne une adresse en mémoire. Un pointeur d'un type donné est déclaré en ajoutant le signe `*` avant le nom du pointeur :

Exemple :

```
int  *pi;      // pointeur sur un entier
char *chaine;  // pointeur sur caractère (chaîne de caractère)
```

1 - 2 - Utilisation

L'opérateur `*` sert à accéder au contenu de la zone mémoire pointée, l'opérateur `->` sert à accéder aux champs d'une structure pointée et l'opérateur `&` sert à obtenir l'adresse de la variable à laquelle il s'applique :

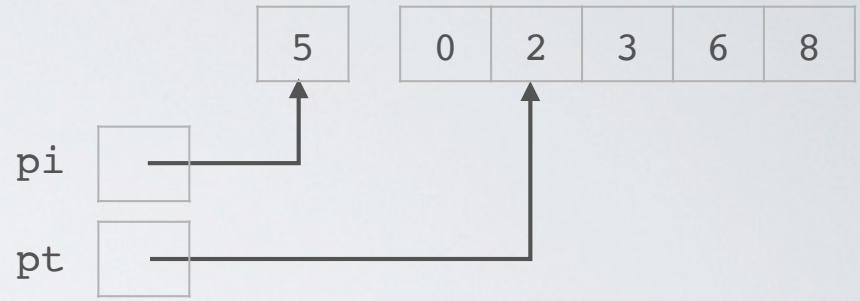
Exemple :

```
int var  = 5,
    t[5] = {0, 2, 3, 6, 8},
    *pi  = NULL,
    *pt  = NULL;
```

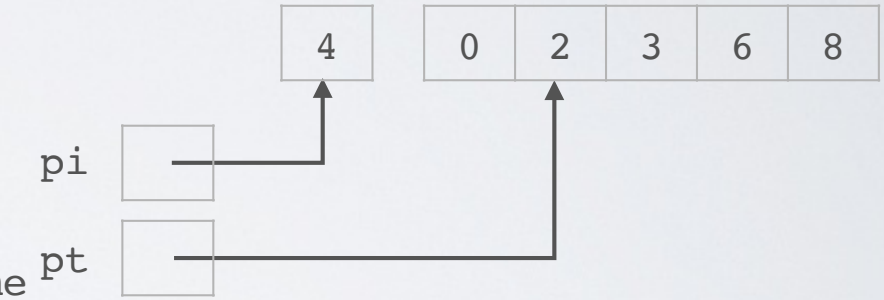
9 - Gestion mémoire

Exemple :

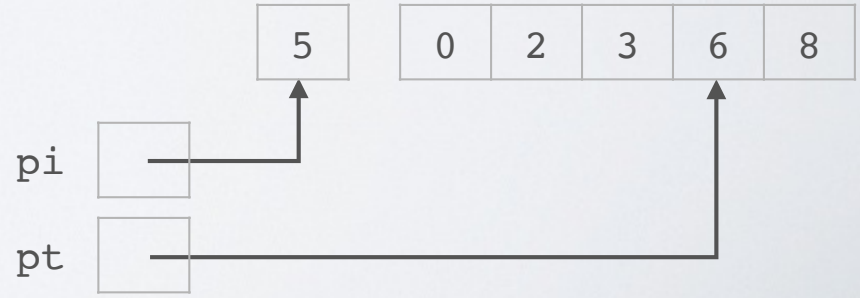
```
pi = &var;              // pi pointe sur var
pt = &t[1];             // pt pointe sur le 2ème
                           élément du tableau
printf("%d\n", *pi);    // affiche 5
```




```
*pi = 4;                // var est modifie
printf("%d\n", var);    // affiche 4
printf("%d\n", *pt);    // affiche 2
```




```
pt += 2;                // pt pointe sur le 4ème
                           élément du tableau
printf("%d\n", *pt);    // affiche 6
```



9 - Gestion mémoire

1 - 3 - Arithmétique sur les pointeurs

Les pointeurs peuvent être incrémentés, décrémentés, additionnés ou soustraits. Dans ce cas, leur nouvelle valeur dépend du type sur lequel ils pointent. Incrémenter un pointeur de `char` ajoute `1` à sa valeur (un caractère est représenté sur 1 octet). Incrémenter un pointeur sur un `int` ajoute `2` ou `4` à sa valeur (cela dépend de l'architecture, un entier pouvant être représenté sur `2` ou `4` octets).
Tout comme avec les tableaux, il est possible d'utiliser les crochets pour accéder au contenu d'un élément pointé. Les écritures suivantes sont équivalentes :

```
int   tab[5],
      *p = tab;
float valFloat,
      *pf;
...
*(p + 1) = 8;
p[1]     = 8;    // identique a la ligne précédente
```

Le nom d'un tableau désigne une constante de type pointeur dont la valeur est l'adresse du premier élément du tableau. Ainsi, un nom de tableau utilisé dans une expression est converti en une adresse du début de ce tableau :

```
p = tab;         // équivalent à p = &tab[0];
```

9 - Gestion mémoire

Les entiers et les réels sont représentés en standard sur 4 octets sur les architectures 64 bits. L'organisation des octets (byte order ou byte sex) dépend, elle, du processeur. Elle peut être qualifiée de petit/grand boutisme (little/big endian).

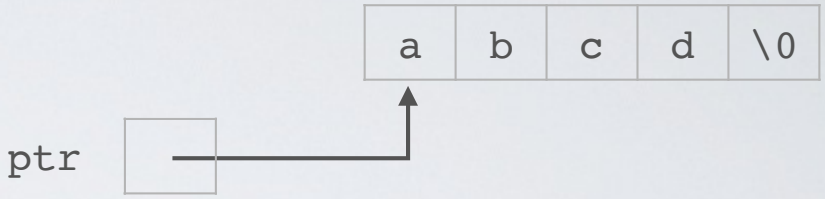
Exemple : organisation des octets en petit boutiste (little endian), c'est à dire les poids forts en dernier (architecture x86).

```
int      valInt;
float    valFloat;
unsigned char *p;
…
p = (unsigned char *) &valInt;
*(p + 0) = 0x7e; // représentation en  complément à 2
*(p + 1) = 0x5c;
*(p + 2) = 0x00;
*(p + 3) = 0x00;
printf("ValInt = %d\n", valInt); // va afficher   23678

p = (unsigned char *) &valFloat;
*(p + 0) = 0x0;  // représentation en IEEEP754
*(p + 1) = 0xd8;
*(p + 2) = 0xbe;
*(p + 3) = 0x44;
printf("ValInt = %f\n", valFloat); // va afficher 1526,75
```

9 - Gestion mémoire

Quand une chaîne littérale est utilisée dans une expression, comme pour un tableau, elle est convertie en une adresse sur son début :

```
char *ptr = "abcd";

...
printf("%s\n", ptr); // affiche la chaîne abcd
```



1 - 4 - Pointeur générique

Les pointeurs de type `void *` sont utilisés pour pointer sur quelque chose dont on ne connaît pas le type. Les seuls opérateurs autorisés avec les pointeurs génériques sont :

- l'affectation
- la conversion de type

9 - Gestion mémoire

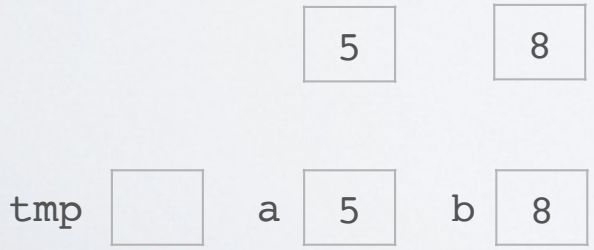
1 - 5 - Passage de paramètres

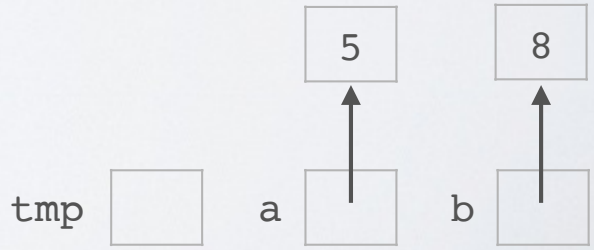
Le seul mode de passage de paramètre en C est le passage par valeur. Ce qui, à priori, interdit la modification de paramètre. En fait, pour réaliser un passage de paramètre modifiable, il suffit de passer une valeur qui est l'adresse du paramètre :

Exemple :

Passage de paramètres	
par valeur	par adresse

`void echange(int a, int b) {`
`  int tmp;`
`  tmp = a;`
`  a  = b;`
`  b  = tmp;`
`}`
	`void echange(int *a, int *b) {`
	`  int tmp;`
	`  tmp = *a;`
	`  *a  = *b;`
	`  *b  = tmp;`
	`}`





9 - Gestion mémoire

Lorsqu'on passe en paramètre un tableau, on passe en réalité l'adresse du premier élément de ce tableau. Les éléments du tableau ne sont pas recopiés. Il s'agit donc d'un passage par adresse. Mais ceci est cohérent avec le fait qu'il n'existe pas de variable désignant un tableau comme un tout.

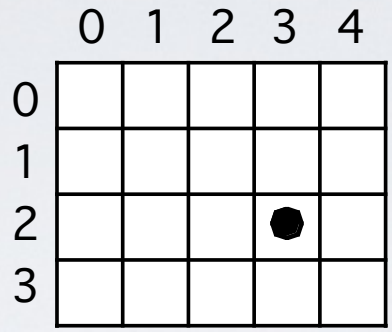
Un tableau à deux dimensions est un tableau unidimensionnel dont les éléments sont des tableaux. La première dimension est celle des lignes et la seconde celle des colonnes. Ceci se généralise aux tableaux à plus de 2 dimensions.
Dans le cas d'un passage en paramètre d'un tableau multidimensionnel, il n'est pas nécessaire de donner la taille de la première dimension, mais il faut préciser les autres si on désire référencer les éléments du tableau par indexation et non en utilisant un pointeur :

Exemple :

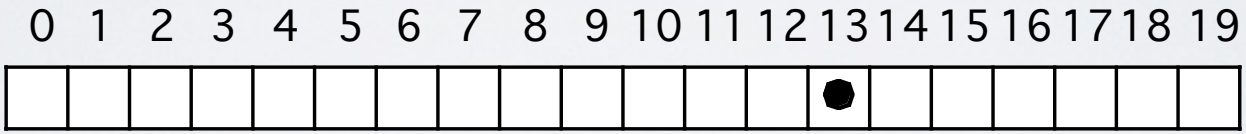
```
// paramètre tableau unidimensionnel
void fonction1(int t[]);

// paramètre tableau bidimensionnel (10 colonnes)
void fonction2(int t[][10]);
```

9 - Gestion mémoire

En effet, l'indexation induit un calcul de la position de l'élément indexé par rapport aux dimensions du tableau.
Pour exemple, considérons un tableau carré 4 X 5, 4 lignes et 5 colonnes.



Le tableau est représenté en mémoire comme un espace linéaire de 20 (4 * 5) éléments :



L'indexation du tableau par `T[2][3]` signifie qu'on souhaite accéder au 13ème élément (2 * 5 + 3). Ainsi, l'absence de déclaration de la seconde dimension empêche le compilateur de générer l'adressage correct.

9 - Gestion mémoire

1 - 6 - Pointeur vers une fonction

Le nom d'une fonction est une constante de type pointeur vers fonction :

```
int f(int x, int y) {        // fonction f
    return x + y;
}

...

int (*pf)(int, int);         // pointeur vers fonction admettant
                             // 2 entiers en paramètres
...
pf = f;                      // pointeur vers la fonction f

printf("%d\n", (*pf)(3,5)); // appel de la fonction
                             // par l'intermédiaire du pointeur
                             // sur fonction
                             // affiche 8
```

9 - Gestion mémoire

2 - Allocation dynamique

2 - 1 - `malloc()`

L'allocation d'une nouvelle zone mémoire se fait par la fonction **`malloc()`**. Elle se fait dans un endroit particulier de la mémoire appelée tas (heap) :

```
void *malloc(size_t taille);
```

L'argument transmis correspond à la taille en octets, de la zone mémoire désirée. La valeur retournée est un pointeur sur la zone mémoire allouée, ou **`NULL`** en cas d'échec de l'allocation.
Le pointeur retourné doit être transformé, à l'aide de l'opérateur de caste **`()`**, en un pointeur sur le type de donnée approprié.

Exemple :

```
int i,
    **T; // pointeur sur un tableau bidimensionnel

// allocation dynamique d'un tableau de dimL lignes et dimC colonnes
T = (int **) malloc(sizeof(int *) * dimL);
for (i = 0; i < dimL; i++) T[i] = (int *) malloc(sizeof(int) * dimC);
```

9 - Gestion mémoire

2 - 2 - `calloc()`

Une deuxième fonction, **`calloc()`**, permet à l'instar de **`malloc()`** une allocation de mémoire :

```
void *calloc(size_t nbElements, size_t tailleElement);
```

Elle sert principalement à allouer de l'espace mémoire pour des tableaux. Le premier argument désigne le nombre d'éléments du tableau et le deuxième, la taille d'un élément.

2 - 3 - `realloc()`

Fréquemment, il s'avère nécessaire de redimensionner un tableau dont l'espace mémoire a été alloué dynamiquement. Ceci se fait par la fonction **`realloc()`** :

```
void *realloc(void *zoneMemoire, size_t taille);
```

Cette fonction se comporte comme **`malloc()`**, à la différence qu'elle recopie le contenu de la zone mémoire actuelle désignée par le premier argument.
Si la nouvelle taille est supérieure à celle de l'ancien bloc mémoire, ce dernier est étendu, son contenu se retrouvant au début du nouveau bloc mémoire.
Si la nouvelle taille est inférieure à celle de l'ancien bloc mémoire, il y a libération de mémoire et le contenu de ce dernier est tronqué.
En cas d'échec de l'allocation, la valeur retournée est **`NULL`** mais, l'ancien bloc mémoire reste inchangé.

9 - Gestion mémoire

3 - Libération mémoire

La mémoire allouée doit, à un moment ou un autre, être libérée. Cette libération mémoire se fait par la procédure **`free()`** :

```
void free(void *pointeur);
```

L'argument passé à cette procédure est l'adresse d'une zone mémoire précédemment allouée par **`malloc()`**, **`calloc()`** ou **`realloc()`**. Une fois qu'une zone mémoire a été libérée, il ne faut sous aucun prétexte y accéder, de même qu'il ne faut pas tenter de la libérer une seconde fois.

Exemple :

```
// Désallocation mémoire du tableau bidimensionnel
for (i = 0; i < dimL; i++) free(T[i]);
free(T);
```

9 - Gestion mémoire

Une autre fonction, **`alloca()`**, permettant une allocation mémoire, ainsi qu'une libération automatique constitue avec le couple **`malloc()`** / **`free()`** une alternative :

```
void *alloca(size_t taille);
```

Le comportement de cette fonction est identique à celui de **`malloc()`**, mais les zones mémoires allouées ne le sont plus dans le tas mais dans la pile. Ainsi, la désallocation se produit au moment ou la fonction dans laquelle la demande d'allocation a eu lieu se termine. Il est évident que l'utilisation de cette fonction doit se faire avec précaution, notamment dans les fonctions récursives afin d'éviter un débordement de pile.

4 - Règles de bonne conduite

Un certain nombre de règles, quand elles sont observées, permettent de se mettre à l'abri de problèmes :

- toute déclaration de pointeur s'accompagne de son initialisation à **`NULL`**,
- avant tout appel de **`malloc()`**, on doit s'assurer que le pointeur à allouer est bien **`NULL`**,
- après tout appel de **`malloc()`**, on s'assure qu'aucune erreur ne s'est produite,
- avant de libérer une zone mémoire, on s'assure que son pointeur n'est pas **`NULL`**,
- dès qu'une zone mémoire vient d'être libérée par **`free()`**, on réinitialise son pointeur à **`NULL`**.

10 - Chaînes de caractères

1 - Conventions

Le type chaîne de caractères n'existe pas en C, dans la mesure où l'on ne peut pas y déclarer des variables d'un tel type. Par contre, il existe une convention de représentation des chaînes qui consiste à les considérer comme une suite de caractères terminée par un octet nul. Cela signifie qu'une chaîne de n caractères occupe n+1 octets en mémoire. Cette convention est utilisée par le compilateur pour représenter les constantes chaînes ainsi que par un certain nombre de fonctions.

2 - Utilisation des chaînes

Les chaînes de caractères peuvent être manipulées de 2 façons :

- en utilisant des fonctions de la bibliothèque standard,
- en manipulant individuellement chaque caractère.

10 - Chaînes de caractères

2 - 1 - Création d'une chaîne

La création d'une chaîne suppose de réserver en mémoire un emplacement. Cette réservation peut se faire de 2 façons :

- de façon statique : c'est le compilateur qui réserve la place en mémoire. Cela suppose que la taille de la chaîne est bornée et ne pourra pas évoluer.

  Exemple :

  ```
  // chaîne d'au plus 30 caractères
  char ch[31];
  ```

- de façon dynamique : la mémoire est allouée dynamiquement en cours d'exécution. De cette façon l'emplacement mémoire peut s'adapter à la taille exacte de la chaîne au moment de l'allocation. Il est toujours possible d'allouer d'autres emplacements ou d'étendre l'emplacement existant en cas de besoin.

  Exemple :

  ```
  // chaîne d'au plus 30 caractères
  char *ch = (char *) malloc(31);
  ```

10 - Chaînes de caractères

2 - 2 - Initialisation d'une chaîne à sa réservation

Exemple :

```
char ch[20] = "Bonjour ",
     *adr   = "Bonjour ",
     *msg   = "le Monde";
```

Bien qu'une chaîne de caractères puisse être vue comme un tableau de caractères, il existe des différences entre la première déclaration et les deux suivantes :

- l'écriture de **`ch = msg;`** est illicite puisque **`ch`** est une constante pointeur qui correspond à l'adresse que le compilateur a attribuée au tableau **`ch`**. De plus, aucune constante chaîne n'apparaît dans la première déclaration et ce, malgré la notation employée ("…"), laquelle ici, n'est qu'une facilité d'écriture remplaçant :

  ```
  char ch[20] = {'B', 'o', 'n', 'j', 'o', 'u', 'r', ' ', '\0'},
  ```

- l'écriture de **`adr = msg;`** est parfaitement licite mais ne signifie nullement que la chaîne **`msg`** est recopiée dans la chaîne **`adr`**. C'est seulement la valeur de la variable **`msg`** (adresse de la chaîne "**`le Monde`**") qui est placée dans la variable **`adr`**. Cette instruction a pour effet de laisser en mémoire la chaîne "**`Bonjour `**" sans aucun référence.

10 - Chaînes de caractères

2 - 3 - Modification d'une chaîne de caractères

Quelle que soit la manière dont on a alloué un emplacement mémoire pour une chaîne de caractère et quel que soit son contenu, on peut lui attribuer une nouvelle valeur, caractère par caractère.

Exemple :

```
char ch[20],
     *adr1,
     *adr2;
…

adr1[5]    = 'A'; // modifie le 6ème caractère de la chaîne adr1
*(ch + 1)  = 'B'; // modifie le 2ème caractère de la chaîne ch
*(adr1 + 2)= 'C'; // modifie le 3ème caractère de la chaîne adr1

…
// recopie de la chaîne adr1 dans adr2
adr2 = (char *) malloc(strlen(adr1)+1);
for (i = 0; i <= strlen(adr1); i++) adr2[i] = adr1[i];
```

10 - Chaînes de caractères

2 - 4 - Affichage d'une chaîne

`printf` avec la séquence d'interprétation **`%s`** permet l'affichage d'une chaîne de caractères.

Exemple :

```
char message[] = "Bonjour le Monde !";
printf(":%s:",    message);
printf(":%5s:",   message);
printf(":%30s:",  message);
printf(":%-30s:", message);
printf(":%.5s:",  message);
```

produit l'affichage suivant :

```
:Bonjour le Monde !:
:Bonjour le Monde !:
:            Bonjour le Monde !:
:Bonjour le Monde !            :
:Bonjo:
```

10 - Chaînes de caractères

3 - Fonctions de manipulation de chaînes

Pour disposer de ces fonctions, il faut inclure dans le programme source le contenu du fichier **`string.h`**.
L'inclusion de ce fichier dans le programme source se fait à l'aide de la directive :

```
#include <string.h>
```

2 - 3 - 1 - Fonctions de copie

`char *strcpy(char *dest, const char *src)`

`strcpy` copie la chaîne **`src`** vers l'adresse **`dest`** y compris l'octet nul de fin de chaîne. Les deux chaînes **`src`** et **`dest`** ne doivent pas avoir de partie commune. **`strcpy`** retourne l'adresse **`dest`**.

`char *strncpy(char *dest, const char *src, size_t longueur)`

`strncpy` copie au maximum longueur caractères de la chaîne **`src`** vers l'adresse **`dest`** en complétant éventuellement avec des octets nuls. Si cette longueur est atteinte, l'octet nul de fin de chaîne n'est pas recopié. Les deux chaînes **`src`** et **`dest`** ne doivent pas avoir de partie commune. **`strncpy`** retourne l'adresse **`dest`**.

`void *memcpy(void *dest, const void *src, size_t longueur)`

`memcpy` copie longueur octets depuis l'adresse **`src`** vers l'adresse **`dest`**. Les deux zones **`src`** et **`dest`** ne doivent pas avoir de partie commune.

`void *memmove(void * dest, const void *src, size_t longueur)`

`memmove` fonctionne comme **`memcpy`** et accepte que **`src`** et **`dest`** se chevauchent.

10 - Chaînes de caractères

2 - 3 - 2 - Fonctions de concaténation

`char *strcat(char *dest, const char *src)`

strcat concatène la chaîne **src** à la chaîne **dest**. Les deux chaînes **src** et **dest** ne doivent pas avoir de partie commune. **strcat** retourne l'adresse **dest**.

`char *strncat(char *dest, const char *src, size_t longueur)`

strncat concatène au maximum longueur caractères de la chaîne **src** à la chaîne **dest**. Les deux chaînes **src** et **dest** ne doivent pas avoir de partie commune. **strncat** retourne l'adresse **dest**.

2 - 3 - 2 - Fonctions de comparaison

`int strcmp(const char *chaine1, const char *chaine2)`

strcmp compare les chaînes **src** et **dest** et retourne :
une valeur négative si **chaine1** < **chaine2**
une valeur positive si **chaine1** > **chaine2**
0 si **chaine1** = **chaine2**

`int strncmp(const char *chaine1, const char *chaine2, size_t n)`

strncmp fonctionne comme **strcmp** mais compare seulement les n premiers caractères.

`int memcmp(const char *zone1, const char *zone2, size_t n)`

memcmp fonctionne comme **strncmp** mais compare des octets et non pas des caractères.

10 - Chaînes de caractères

2 - 3 - 2 - Fonctions de recherche

`char *strchr(const void *chaine, int c)`

`strchr` retourne l'adresse de la première occurrence du caractère **`c`** dans la chaine ou **`NULL`** sinon.

`char *strrchr(const char *chaine1, int c)`

`strrchr` retourne l'adresse de la dernière occurrence du caractère **`c`** dans la chaine ou **`NULL`** sinon.

`size-t *strspn(const char *chaine1, const char *chaine2)`

`strspn` retourne la longueur du segment initial de **`chaine1`** contenant l'un des caractères de **`chaine2`**.

`size-t *strcspn(const char *chaine1, const char *chaine2)`

`strcspn` retourne la longueur du segment initial de **`chaine1`** ne contenant aucun des caractères de **`chaine2`**.

`char *strpbrk(const char *chaine1, const char *chaine2)`

`strpbrk` retourne l'adresse de la première occurrence dans la chaine **`chaine1`** de n'importe quel caractère de la chaine **`chaine2`**, ou **`NULL`** sinon.

`char *strstr(const char *chaine1, const char *chaine2)`

`strstr` retourne l'adresse de la première occurrence dans la chaine **`chaine1`** de la chaîne **`chaine2`**, ou **`NULL`** sinon.

`char *strtok(const char *chaine, const char *delimiteurs)`

`strtok` éclate la chaine en sous-chaînes délimitées par l'un des caractères de delimiteurs. A chaque appel **`strtok`** retourne l'adresse d'une sous-chaine, ou **`NULL`** sinon. Les appels suivants à **`strtok`** doivent se faire avec une valeur **`NULL`** pour **`chaine`**.

10 - Chaînes de caractères

`void *memchr(const void *zone, int c, size_t n)`

memchr fournit l'adresse de la première occurrence de la valeur **c** dans les **n** octets d'adresse **zone** ou **NULL** sinon.

2 - 3 - 3 - Fonctions de diverses

`size-t *strlen(const char *chaine)`

strlen retourne la longueur de la **chaine**.

`void *memset(void *zone, int c, size_t n)`

memset copie **n** fois le caractère **c** dans la **zone**.

`char *strerror(int numErreur)`

strerror retourne l'adresse de la chaîne correspondant au message d'erreur **numErreur**.

11 - Le préprocesseur

1 - Présentation

La génération de code à partir d'un fichier source en C se déroule en 2 phases:

- le prétraitement.
- la compilation.

On parle de préprocesseur pour désigner le programme en charge de la première phase. Son travail consiste en un traitement du fichier source fondé sur l'interprétation d'instructions particulières appelées directives. Il permet de faire :

- de la substitution de source,
- de l'inclusion de source contenu dans un fichier
- de la compilation conditionnelle

2 - Directives

2 - 1 - Définition de symboles et de macros

La directive **`#define`** offre 2 possibilités de définition de nature quelque peu différente :

- la définition de symbole
  `#define identificateur chaîne_quelconque`
- la définition de macro
  `#define identificateur(liste_d'identificateurs) corps_de_macro`

11 - Le préprocesseur

2 - 2 - Directives de compilation conditionnelle

Des directives permettent d'incorporer ou d'exclure des portions du fichier source dans le texte qui sera analysé.

- Compilation conditionnelle fondée sur l'existence de symboles
  A la rencontre des lignes de contrôle suivantes :

```
#ifdef identificateur
suite_de_lignes_1
[#else
suite_de_lignes_2]
#endif
```

La `suite_de_lignes_1` est compilée si `identificateur` a été défini par un `#define identificateur`.

Sinon, la `suite_de_ligne_1` est ignorée et la `suite_de_ligne_2` est compilée si elle existe.

Dans le cas de
`#ifndef identificateur`

la condition est inversée.

11 - Le préprocesseur

- Compilation conditionnelle fondée sur des expressions
  La directive se fonde ici uniquement sur une expression entière que le préprocesseur est capable d'évaluer. Cette dernière peut faire appel à un opérateur unaire particulier defined, connu uniquement du préprocesseur. Cet opérateur fournit la valeur **1** si son opérande est définie et **0** sinon.

```
#if expression
suite_de_lignes_1
[#else
suite_de_lignes_2]
#endif
```

  L'évaluation de **expression** fournit un résultat vrai si la valeur délivrée par **expression** est différente de **0** et faux sinon.
  La **suite_de_lignes_1** est compilée si la condition est vraie. Sinon, la **suite_de_ligne_1** est ignorée et la **suite_de_ligne_2** est compilée si elle existe.

Exemple :

```
#define DEBUG
#if (CODE==1) && defined(DEBUG)
/* lignes 1*/
#endif
```

11 - Le préprocesseur

2 - 3 - Directive d'inclusion de fichier source

```
#include nom_de_fichier
```

Cette directive provoque le remplacement de cette ligne par le contenu du fichier **`nom_de_fichier`**.
La recherche du fichier en question s'effectue dans 2 espaces différents :

- l'espace utilisateur
- l'espace système
  selon que nom_de_fichier est entouré par les caractères " et " ou par les caractères < et >.

Fin